

INF1015 – Programmation orientée objet avancée

Travail dirigé No. 1

Intro au C++ : Types, Structures de décision et répétition, lecture de fichiers, tableaux et structures.

Objectifs : Permettre à l'étudiant de se familiariser avec les différentes structures du langage C++, les fichiers textes, la manipulation des tableaux statiques ainsi que les enregistrements (`struct`).

Durée : Une demie semaine de laboratoire.

Remise du travail : vendredi 15 mai 2026, avant 23 h 30.

Travail préparatoire : « Leçon: introduction à Visual Studio » sur Moodle et lecture des exercices.

Seulement 3 exercices seront corrigés (mais vous devez les faire tous). Le barème de correction est présenté sur Moodle.

Directives particulières

- Utilisez le principe DRY (Don't Repeat Yourself). Considérez utiliser une boucle si vous devez exécuter plusieurs fois la même instruction. À chaque endroit où vous remarquez une duplication de code (vous avez écrit les mêmes opérations plus d'une fois) et qu'il n'est pas possible de l'éliminer avec les structures vues, indiquez-le en commentaire. Vous pouvez ajouter des fonctions autres que celles décrites dans l'énoncé, dans le but d'améliorer la lisibilité.
 - Respecter le guide de codage, les points pertinents pour ce travail sont donnés en annexe à la fin.
 - Faites attention à votre choix de structures de répétition. Rappelez-vous qu'une boucle `for` est utilisée lorsque le nombre d'itérations est connu au moment où elle commence, au contraire de la boucle `while` (ou `do/while`).
 - Utilisez les structures de données appropriées (tableaux, « `struct` »).
 - Vos programmes doivent fonctionner avec les fichiers fournis, mais également avec tout autre fichier qui respecterait le format requis.
 - Compiler avec /W4. Lors de la compilation avec Visual Studio 2022 (ou 2026), votre programme ne devrait pas afficher d'avertissements (« warnings ») de « build ».
 - Les entrées/sorties au clavier/écran doivent être faites avec messages significatifs et vous n'avez pas à valider les entrées sauf si indiqué dans la question.
-

1. **Nombres premiers :** Écrivez un programme qui affiche le premier nombre premier supérieur ou égal à une valeur entrée. Vous devez d'abord lire du clavier un nombre entier puis, à partir de ce nombre, trouver le prochain nombre premier. On assume que le nombre entré sera supérieur ou égal à 2 (pas de vérification nécessaire). Vérifiez si un nombre est premier ou non dans une **fonction**. Le programme doit recommencer jusqu'à ce que l'utilisateur entre la valeur -1.

Pour déterminer si un nombre n est premier, vous pouvez vous baser sur l'algorithme suivant : pour chaque i de 2 à la racine carrée de n , si n modulo i est zéro, i est un diviseur donc ce n'est pas un nombre premier; si la boucle termine sans trouver de diviseur, c'est un nombre premier.

Exemple d'affichage (entrées de l'utilisateur soulignées) :

```
Entrez un nombre entier : 8
Le prochain nombre premier est 11.
Entrez un nombre entier : 23
Le prochain nombre premier est 23.
Entrez un nombre entier : -1
```

2. **Devinette** : Écrivez un programme qui permet de deviner la valeur d'un nombre entier choisi aléatoirement entre 0 et 1000. L'utilisateur doit entrer des essais et le programme affiche si l'essai est inférieur, supérieur ou égal à la valeur choisie aléatoirement. À la fin, le programme indique le nombre de tentatives utilisées pour deviner la valeur. Écrivez une **fonction** permettant de lire une valeur réelle dans un intervalle. La fonction doit à la fois afficher pour demander, vérifier que la valeur est valide et redemander une entrée tant qu'elle ne l'est pas; elle n'a pas à vérifier que ce qui est entré est un nombre, seulement que le nombre est bien dans l'intervalle. Ses paramètres sont le texte à afficher, le min, le max. L'entrée de l'entier de l'utilisateur sera par cette fonction, où le réel résultant sera converti en entier.

Exemple d'affichage (entrées de l'utilisateur soulignées) : (le nombre choisi aléatoirement est 45)

Entrez un nombre entier : -1 *//(à noter que 1001 va produire le même résultat)*

Entrez un nombre entier : 5

Trop bas.

Entrez un nombre entier : 45

Bravo! Vous avez réussi en 2 tentatives!

3. **Collision** : Écrivez un programme qui détermine si une collision se produit dans les x premières secondes d'un trajet. Le temps total (nombre entier supérieur à 0) est fourni par l'utilisateur. Vous devez ensuite lire la position initiale en x (quelconque) et la vitesse en x (entre -100 et 100) de deux trains. Toutes les valeurs doivent être lues avec la **fonction** définie dans l'exercice 2. Vous pouvez utiliser la constante `INFINITY` de `cmath` lorsqu'il n'y a pas de borne supérieure ou inférieure pour l'intervalle. Puis, vous devez indiquer l'emplacement à chaque seconde des deux trains jusqu'à arriver au temps total ou à la première collision. Vous devez finalement indiquer après combien de secondes il y a eu une collision ou indiquer qu'il n'y en a pas eu. Le but est de vérifier à chaque seconde s'il y a une collision, et non de calculer analytiquement le moment de la collision.

Exemple d'affichage (entrées de l'utilisateur soulignées) :

Entrez le temps total : 3

Entrez la position initiale du premier train : 5

Entrez la vitesse du premier train : 200

Entrez la vitesse du premier train : 2

Entrez la position initiale du second train : 15

Entrez la vitesse du second train : -5

Après 1 seconde(s)

Train 1 : 7

Train 2 : 10

Après 2 seconde(s)

Train 1 : 9

Train 2 : 5

Il y a eu une collision pendant la 2e seconde!

4. **Tri** : Écrire un programme qui saisit un tableau d'entiers (pas de vérification nécessaire) de taille 10 et le trie. N'utilisez pas les fonctions de tri qui existent en C++ (qui seront vues plus tard dans la session). Le tri est fait en trouvant le plus petit élément entre les cases 0 et 9 et en échangeant ce minimum avec la case 0. Ensuite, il doit trouver le plus petit élément entre les cases 1 et 9 et échanger ce minimum avec la case 1 et ainsi de suite. Le programme doit ensuite afficher le tableau trié.

Exemple d'affichage (entrées de l'utilisateur soulignées) :

Entrez 10 nombres entiers : 8 4 12 58 3 9 7 8 11 1

Voici le tableau trié : 1 3 4 7 8 8 9 11 12 58

5. **Magasin** : Écrire un programme de traitement de données pour une épicerie (aliment, type, quantité, prix unitaire). Le programme doit créer un tableau de **structures**, y placer l'inventaire en le lisant d'un fichier, et ensuite afficher dans une **fonction** l'aliment le plus cher dans le format suivant :

quantité nom (type) a prix\$ chacun

Un fichier « inventaire.txt » vous est fourni, où chaque ligne contient un aliment sous la forme :

nom → type → quantité → prix ↵

Le *→* est le caractère de tabulation indiqué par « \t » en C/C++, et *↵* est le caractère de fin de ligne « \n ». Les champs *nom* et *type* peuvent contenir des espaces, le document Moodle « Exemples/annexe pour entrées-sorties avec les "streams" C++ » montre des exemples de lecture de ce type. Vous pouvez supposer que l'inventaire contiendra au maximum 10 aliments (constante définie dans votre programme) et que le nom du fichier est toujours « inventaire.txt ».

Annexe 1 : Points du guide de codage à respecter

Les points du guide de codage à respecter impérativement pour ce TD sont les suivants :

(Voir le guide de codage sur le site Moodle du cours pour la description détaillée de chacun de ces points)

- 3 : noms des variables et constantes en lowerCamelCase
- 14, 52 : portée des variables (noms courts/longs)
- 21 : pluriel pour les tableaux (int nombres[;])
- 22 : préfixe n pour désigner un nombre d'objets (int nElements;)
- 24 : variables d'itération i, j, k (pour des indices) mais jamais l
- 25 : is/est pour booléen
- 27 : éviter les abréviations (les acronymes communs doivent être gardés en acronymes)
- 29 : éviter la négation dans les noms
- 33 : entête de fichier
- 42 : include au début
- 46 : initialiser à la déclaration
- 47 : pas plus d'une signification par variable
- 51 : test de 0 explicite (if (nombre != 0))
- 53-54 : boucles for et while
- 58-61 : instructions conditionnelles
- 62 : pas de nombres magiques dans le code
- 63-64 : « double » toujours avec au moins un chiffre de chaque côté du point (i.e. 1.0, 0.5)
- 67-78, 88 : indentation du code et commentaires
- 79,81 : espacement et lignes de séparation
- 83-84 : aligner les variables lors des déclarations ainsi que les énoncés
- 85 : mieux écrire du code incompréhensible plutôt qu'y ajouter des commentaires
- 87 : préférer // pour les commentaires